

MOATLY

Backend — Technical Breakdown

The Moat service and the linkage service: two small Go programs that hold the server-blind story together. The Moat service is the heart; the linkage service is the side-door for email and the Unity app.

Prepared for: Engineering lead, Moatly
Document version: 0.1 · Status: Draft for review
Date: April 2026

0. How to read this document

This breakdown covers the **server side** of Moatly — two Go programs plus their shared database and deletion workflow. Together they are the backend that the Moat mobile app, the Unity ritual app (via the linkage path), and the online portal speak to.

The backend is the largest single piece of the system. It is *also* the most sensitive. This is where the Build Brief's central engineering promise — that we hold no plaintext and cannot be compelled to produce what we do not have — either succeeds or quietly fails. Every other piece of Moatly is downstream of the backend either keeping that promise or breaking it.

Two programs, deliberately split:

- The **Moat service** is the server-blind routing and retention layer. It never sees plaintext. It stores ciphertext blobs, runs the seven-day deletion pipeline, orchestrates Megolm session rotation at cycle boundaries, handles the complaint pipeline metadata-only, and authenticates device challenges.
- The **linkage service** is the small piece that talks to the outside world. It holds account identifiers, email addresses, and Unity-app linkage tokens. It receives prescription envelopes from the Unity app and forwards them via Postmark. It does not talk to the Moat service except through a single account-id-only endpoint.

Splitting them matters. They have *different* threat models. The Moat service is designed never to hold plaintext; the linkage service handles plaintext email content in transit. Keeping them in different processes, different databases, and different ops rotations makes that difference visible in the architecture.

0.1 What this document assumes

- The reader has read the Build Brief v1.0, especially §3 (system architecture), §4 (technology stack), §6 (cryptography), §7 (data model and retention), §8 (API surface), and §9 (the seven-day deletion system). Those sections are the source; this document is the implementation.
- The reader has read the Moat app breakdowns (Phases 1–3). The endpoints defined in those breakdowns are the client-side contract this document implements.
- The reader has read the portal breakdown. The portal has its own small Go backend, specified there; this document does not cover it.

0.2 What this document does not cover

- The Moat mobile app. Covered by Moat_App_Technical_Breakdown_v0.1 and its Phase 2 and Phase 3 siblings.
- The Unity ritual app. Separate build, separate team.
- Firmware. Separate breakdown.
- The portal's own backend (waitlist, device checkout). Covered in the portal breakdown.
- Manufacturing, shipping, commercial operations.

0.3 Document map

§	Contents	Primary reader
1	What the backend is — scope, service split, relationship to everything else.	Lead, founder
2	Decisions required before Week 1.	Lead, security
3	Service architecture — deployment topology, request flows.	All
4	The Moat service — handlers, workers, internal design.	Backend engineers
5	The linkage service — handlers, email-relay operation.	Backend engineers
6	The seven-day deletion pipeline — server side, in full.	Lead, security, SRE
7	Data model and retention — complete Postgres DDL.	Backend + security
8	Full API surface — every endpoint with request and response shape.	Backend + mobile
9	Server-side cryptography — Megolm group state, device attestation, JWT.	Security, backend
10	Observability — the PII-free log and metrics pipeline.	SRE
11	Security and operational posture — secrets, deploys, lawful orders.	Lead, SRE, legal
12	Testing — adversarial tests for the server-blind and retention properties.	QA, security
13	Implementation plan — 10 months, mapped to Build Brief phases.	Lead, PM
14	Open questions.	Lead, founder
15	Post-launch — what the ongoing operational shape looks like.	Lead, SRE

1. What the backend is

1.1 Two programs, one story

The backend is two Go programs that deploy, deploy, observe, and fail independently. They share a Postgres cluster for convenience (same operational plane, same backups) but use different schemas so the blast radius of a compromise to either service is bounded.

Program	What it is	What it holds
moat-service	Go, single statically-linked binary. Deployed to ECS Fargate in AWS eu-central-1 (Frankfurt). Stateless across the fleet; state is in Postgres and Temporal.	Ciphertext only. Opaque account IDs. Device attestation pubkeys. Moat group state (Megolm session identifiers, membership rosters). Complaint metadata (never content). Deletion audit log.
linkage-service	Go, single binary. Same ECS cluster, different service definition. Runs 2 replicas behind an internal ALB.	Account IDs (shared with moat-service, read-only). Email addresses. Unity linkage tokens. Postmark relay receipts. Never Moat content. Never ritual choices.

1.2 What 'the backend' is responsible for

- **Authenticating devices** via attestation + composite identity code. No passwords.
- **Routing encrypted messages** between Moat members. Server sees ciphertext; clients decrypt.
- **Retaining messages for seven days** and then making them verifiably unrecoverable.
- **Forming Moats** — placing users into 8–12 member groups on a FIFO basis at formation time.
- **Running the cycle clock** — every Moat is on a 7-day rhythm; the server owns the clock.
- **Orchestrating session rotation** at cycle boundaries and on membership changes.
- **Handling complaints** at the metadata level — no content ever read — and triggering silent reassignment at thresholds.
- **Relaying email prescriptions** from the Unity ritual app via Postmark. No storage of body.
- **Publishing signed firmware manifests** for device OTA.
- **Producing operational metrics** that are PII-free by construction, for the status page and transparency reports.

1.3 What the backend deliberately does not do

- No decryption of Moat content. Not in handlers, not in workers, not in debug endpoints.
- No storage of plaintext content past the point of Postmark hand-off.
- No analytics on user behaviour. Infrastructure metrics only.
- No ranking. No recommendations. No graph of who-said-what-to-whom.

- No cross-Moat queries. No 'all messages by this user ever' endpoint.
- No 'admin impersonation' path. There is no god-mode.

1.4 Who calls the backend

Caller	Talks to	Why
Moat mobile app	moat-service + linkage-service	All of Moat gameplay + account management.
Unity ritual app	linkage-service only	Prescription relay endpoint.
Online portal	Neither (it has its own small backend)	The portal is a sibling, not a dependent, of the Moat backend.
Firmware (OTA)	moat-service (signed manifest endpoint)	Fetching the latest signed firmware.
Ops tooling	Both, via authenticated internal paths	Deploys, health, transparency metrics.

2. Decisions required before Week 1

Nine decisions. Default recommendation, trade-off summary, owner.

2.1 Go framework — net/http + chi, or gin/echo/fiber

Recommendation: net/http + chi for routing. Zero framework magic.

Go's standard library is sufficient for HTTP handling at our scale. chi adds method+path routing and a middleware pipeline without pulling in reflection or an ORM. gin/echo/fiber are larger surfaces — more middleware, more hooks, more places where a telemetry feature might ship by default and need to be disabled. The whole backend is small enough that a thin router is preferable to a framework.

2.2 ORM — sqlc, database/sql + scany, or plain database/sql

Recommendation: sqlc. Hand-written SQL in .sql files, generated type-safe Go accessors.

sqlc reads SQL files and generates Go types. We write the schema we want, the queries we want, and sqlc produces the typed Go functions that execute them. No reflection. No ORM abstraction leaking. Every database touch is a plain SQL statement visible in code review.

Alternatives: GORM pulls in reflection and auto-migration magic we do not want in a service whose schema changes are security-relevant. database/sql + scany is fine for smaller surfaces but trades type safety for line count.

2.3 WebSocket library — gorilla/websocket, nhooyr/websocket, or stdlib

Recommendation: nhooyr/websocket (now github.com/coder/websocket).

nhooyr is a context-aware, lean implementation. gorilla/websocket is deprecated as of 2022 (still works but not actively maintained). The standard library has no WebSocket support. nhooyr gives us clean goroutine lifetimes, proper context cancellation, and a small API surface.

2.4 Workflow engine — Temporal, cron + Postgres locks, or bespoke

Recommendation: Temporal, self-hosted in the same cluster.

The deletion pipeline is a workflow: multiple steps, must be durable across process restarts, must retry on failure, must produce an auditable history of every run. Temporal gives us all of this out of the box. The alternative — cron jobs with Postgres advisory locks plus a retry loop — would reproduce the same features by hand and still be harder to audit.

Self-hosted rather than Temporal Cloud because Temporal Cloud is a US-based SaaS and the brief's EU-first posture (§4.2) applies. Self-hosting is one more thing to operate, but it is a well-documented thing.

2.5 Server-side vodozamac — full crate or parser-only

Recommendation: Parser-only. The server parses Megolm envelope metadata (session_id, message_index) but never holds decryption keys.

The Build Brief (§4.2) says 'vodozamac (Olm/Megolm) behind a narrow FFI (server-side protobuf wire format only)' — server-side we are blind to contents. We still need to parse the Megolm envelope to extract `session_id` and `message_index` for dedupe, ordering, and session-rotation integrity checks. We do this with a small Go wrapper around vodozamac's envelope parsing primitives. No key material ever lives in server memory.

2.6 Secrets management

Recommendation: AWS KMS for all runtime secrets. SOPS with KMS for configuration in the repo.

Runtime secrets (Postgres password, Stripe keys, Postmark token, device-signing private key for OTA) are fetched from KMS-encrypted Secrets Manager at service startup. Nothing in plain environment variables. SOPS-encrypted YAML in the repo stores non-sensitive config that benefits from being in-repo for change tracking.

2.7 Observability — Grafana Cloud, self-hosted Grafana, or Datadog

Recommendation: Grafana Cloud with the EU region selected. Metrics via OpenTelemetry → Grafana Cloud Prometheus. Logs via Loki. Alerting via Grafana OnCall.

The Build Brief (§4.2) names Grafana Cloud. This matches. Datadog is the market leader but is US-based and has a richer tracking posture than we want in our ops provider. Self-hosting the full Grafana stack is possible but is more than a one-SRE operation.

2.8 Load balancer — ALB, NLB, or API Gateway

Recommendation: AWS Application Load Balancer (ALB) with AWS WAF attached.

ALB terminates TLS, integrates with AWS Certificate Manager for renewal, and supports WebSocket natively. API Gateway adds a request-per-month cost that does not help us. NLB is faster at L4 but lacks the L7 features (path-based routing, header manipulation, WAF integration) we want.

2.9 Database failover — RDS Multi-AZ or self-managed

Recommendation: RDS Postgres 16 Multi-AZ in eu-central-1, with a read replica in eu-west-1 (Dublin) per brief §4.2.

RDS Multi-AZ gives us synchronous replication to a standby with automatic failover. The Dublin replica is the brief-mandated failover region. Self-managed Postgres would save some cost but is not what our small SRE team should be operating.

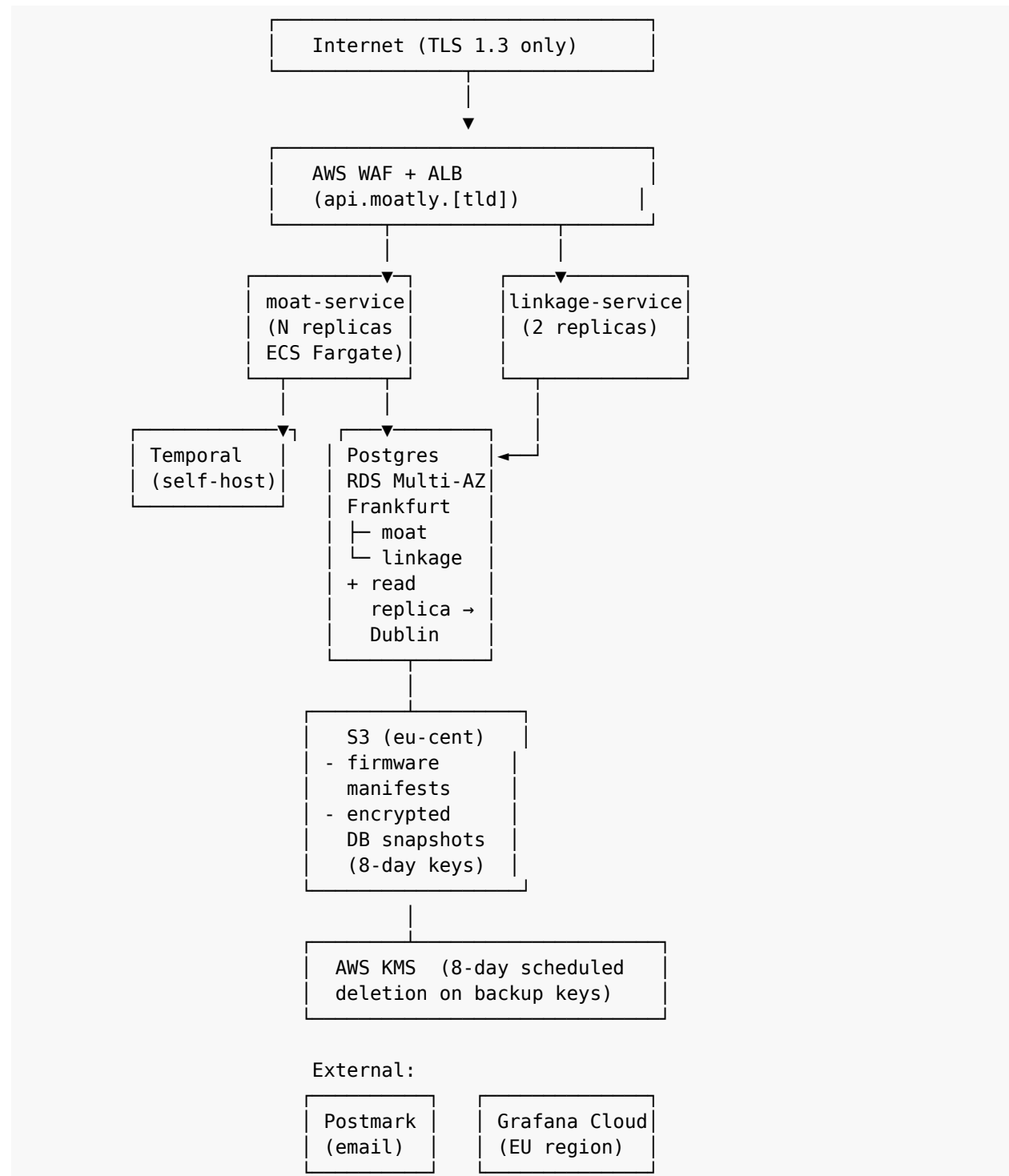
2.10 Summary table

Decision	Default recommendation	Owner
HTTP framework	net/http + chi	Lead
Database layer	sqlc — hand-written SQL, generated types	Lead
WebSocket library	nhooyr / coder/websocket	Lead

Decision	Default recommendation	Owner
Workflow engine	Temporal, self-hosted	Lead + SRE
Server-side vodozemac	Parser-only; no server-side keys	Security + lead
Secrets	AWS KMS + Secrets Manager; SOPS for repo	SRE
Observability	Grafana Cloud EU	SRE
Load balancer	ALB + WAF	SRE
Database	RDS Postgres 16 Multi-AZ Frankfurt, Dublin read replica	SRE

3. Service architecture

3.1 Deployment topology



3.2 Request flow — a typical whisper post

To show how the pieces fit together, here is what happens when a user posts a whisper in their Moat:

1. Moat app ensures a fresh JWT (cached or via device handshake).
2. POST /v1/moats/current/messages
 - Authorization: Bearer <jwt>
 - Content-Type: application/x-megolm-group-message
 - Body: Megolm-encrypted envelope (ciphertext)
3. ALB → moat-service replica.
4. moat-service middleware chain:
 - a. TLS-terminated request → request ID assigned.
 - b. JWT validated (signature, expiry, audience).
 - c. Rate limit checked (per-account token bucket).
 - d. Body size capped at 64KB (whisper + overhead).
5. Handler:
 - a. Parse Megolm envelope metadata (session_id, msg_index).
 - b. Verify session_id belongs to the JWT's Moat + cycle.
 - c. Verify message_index is monotonic (dedupe + anti-replay).
 - d. INSERT into moat_messages (ciphertext, posted_at, expires_at).
 - e. Emit 'new_message' to the Moat's WebSocket broadcaster.
 - f. Return 201 with message_id, posted_at, expires_at.
6. Broadcaster fanout:
 - Every WebSocket connection for this moat_id receives the ciphertext as a framed message. Recipients decrypt on-device.

At no point does moat-service decrypt, read, or log the ciphertext.

3.3 Trust boundaries

Boundary	What crosses, and in what direction
Client ↔ ALB	TLS 1.3. Client IP logged at the WAF layer for rate-limit purposes; stripped before any downstream log entry.
ALB ↔ moat-service	JWT-authenticated requests. Moat message bodies are opaque ciphertext. Outer envelope is JSON (protobuf for message POSTs).
ALB ↔ linkage-service	Either JWT (from Moat app) or Unity linkage token (from Unity app). Prescription bodies cross as plaintext briefly on their way to Postmark; never stored.
moat-service ↔ Postgres	Connection pool, per-service credentials. Roles: moat_rw, moat_ro_deletion_audit. No superuser.
linkage-service ↔ Postgres	Separate credentials: linkage_rw. Cannot read any moat-service schema.
moat-service ↔ Temporal	Workflows submitted; Temporal worker runs deletion workflows. Temporal has no access to Postgres rows.
service ↔ AWS KMS	Only for secret decryption at startup and backup-key rotation.

Boundary	What crosses, and in what direction
service ↔ Postmark	API key in KMS. No inbound from Postmark except delivery webhooks for audit.

3.4 What each program knows about the other

- moat-service and linkage-service share only the account_id space.
- moat-service never reads from linkage-service's tables.
- linkage-service never reads moat-service's tables, except for a single 'does this account exist' check via a signed internal API.
- Neither service calls the other's endpoints in a request path. Their only cross-service contact is the 'does this account exist' check, which is a one-way linkage → moat-service call.

4. The Moat service

The Moat service is the largest and most security-sensitive of the two programs. It is structured as a set of handlers (HTTP and WebSocket) plus a set of workers (Temporal workflows and periodic jobs). This section specifies each.

4.1 Package layout

```
cmd/
├── moat-service/
│   └── main.go                # wires everything together
├── moat-migrator/
│   └── main.go                # applies schema migrations

internal/
├── auth/
│   ├── challenge.go          # /v1/auth/challenge
│   ├── verify.go             # /v1/auth/verify (composite code)
│   ├── refresh.go            # /v1/auth/refresh
│   ├── jwt.go                # sign, verify, claims
│   └── softauth.go           # DELETED at Phase 3 T-0 cutover
├── accounts/
│   ├── create.go
│   ├── delete.go
│   └── email.go               # set/remove email → linkage-service call
├── devices/
│   ├── enrol.go
│   ├── tombstone.go
│   └── revoke.go
├── moats/
│   ├── placement.go          # FIFO-based Moat formation
│   ├── join.go
│   ├── state.go              # GET /v1/moats/current
│   ├── session.go            # session reset, Megolm orchestration
│   └── cycle.go               # cycle clock + boundary broadcaster
├── messages/
│   ├── post.go               # POST message handler
│   ├── envelope.go           # Megolm envelope parse (vodozemac)
│   └── broadcast.go           # WebSocket fanout
├── ws/
│   ├── gateway.go            # nhooyr upgrade + per-moat broadcaster
│   ├── ticket.go             # short-lived WS ticket exchange
│   └── frames.go              # outbound frame typing
├── complaints/
│   ├── file.go
│   ├── thresholds.go         # evaluate + trigger reassignment
│   └── serial.go              # detect weaponisation (5+ in 90d)
├── firmware/
│   └── manifest.go            # serve signed OTA manifest
├── deletion/
│   ├── workflow.go           # Temporal workflow definition
│   ├── activities.go          # individual deletion activities
│   └── audit.go               # append-only audit log
```

```

├─ observability/
│   ├── logger.go          # PII-stripping logger
│   ├── metrics.go         # OTel metrics
│   └── trace.go           # OTel tracing (no body spans)
├─ db/
│   ├── queries/*.sql      # every query as a .sql file
│   └── generated/         # sqlc output
├─ httputil/
│   ├── middleware.go      # auth, rate-limit, request-id
│   └── errors.go          # typed error responses

```

4.2 HTTP handlers — overview

Sixteen handlers across eight packages. Each handler is a single Go function of the shape `func(http.ResponseWriter, *http.Request)` that calls into a package function returning typed results.

Package	Handlers	Responsibility summary
auth	challenge, verify, refresh, (softauth_* — removed)	Exchange attested device signatures for JWTs.
accounts	create, delete, set_email, remove_email	Opaque account lifecycle.
devices	enrol, revoke, tombstone	Device attestation binding and revocation.
moats	join, current_state, session_reset	Moat lifecycle and membership.
messages	post_message, stream (WebSocket)	Ingest ciphertext; fan out to members.
complaints	file, list_mine	Complaint receive + metadata-only review.
firmware	get_manifest	Serve signed OTA manifest.
internal ops	deletion_status, transparency_metrics, health	SRE + portal status-page inputs.

4.3 Handler template

Every handler follows the same shape — this keeps them short and auditable.

```

func (s *Service) PostMessage(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()

    // 1. Authenticated? (Middleware has validated JWT; claims attached.)
    claims := auth.MustClaims(ctx)

    // 2. Rate limit (typed key, per-account bucket).
    if err := s.RateLimiter.Check(ctx, rate.KeyAccount(claims.AccountID)); err != nil {
        httputil.WriteError(w, http.StatusTooManyRequests, err)
        return
    }
}

```

```
// 3. Body size / type checks.
body, err := httputil.ReadCappedBody(r, MessageSizeLimit)
if err != nil {
    httputil.WriteError(w, http.StatusRequestEntityTooLarge, err)
    return
}

// 4. Domain call – all business logic lives here.
msg, err := s.Messages.Post(ctx, messages.PostParams{
    AccountID: claims.AccountID,
    MoatID:    claims.MoatID,
    Cycle:     claims.CycleNumber,
    Envelope:  body,
})
if err != nil {
    httputil.WriteDomainError(w, err)
    return
}

// 5. Response.
httputil.WriteJSON(w, http.StatusCreated, messages.PostResponse{
    MessageID: msg.ID,
    PostedAt:  msg.PostedAt,
    ExpiresAt: msg.ExpiresAt,
})
}
```

4.4 Moat formation — the placement worker

The brief (§5.4.1) specifies Moat formation: 'FIFO of users who completed onboarding. No affinity matching. No demographic clustering.' The placement worker is the concrete implementation.

Every 60 seconds, a Temporal-scheduled activity runs:

- **Read the placement queue** (ordered insert of `account_ids` that completed onboarding).
- **Take the oldest 10** (target group size). If the queue has fewer than 8, wait for the next tick.
- **Create a new Moat** with a new UUID.
- **Emit a 'formed' event** that triggers cycle-zero session creation.
- **Clients in that Moat receive a WebSocket notification** that their Moat is formed; they begin session-key exchange.

No affinity matching. Ever. The placement code has no access to any user-descriptor data because no such data exists.

4.5 Cycle clock

Every Moat has a `cycles_ends_at` timestamp. The cycle clock is a single Temporal workflow that sleeps until that timestamp, then triggers the cycle-rotation cascade.

```
// Temporal workflow – one instance per active Moat.
func CycleWorkflow(ctx workflow.Context, input CycleInput) error {
```

```

    state, err := workflow.ExecuteActivity(ctx, LoadMoatState, input.MoatID).Get(ctx,
nil)
    if err != nil {
        return err
    }

    workflow.Sleep(ctx, time.Until(state.CyclesEndsAt))

    // 1. Broadcast SessionReset to all members via WebSocket.
    if err := workflow.ExecuteActivity(ctx, BroadcastSessionReset,
input.MoatID).Get(ctx, nil); err != nil {
        return err
    }

    // 2. Wait for the deletion pipeline to complete for this Moat.
    if err := workflow.ExecuteActivity(ctx, RunDeletionPass, input.MoatID).Get(ctx,
nil); err != nil {
        return err
    }

    // 3. Create the new cycle record; reset membership for the new session.
    if err := workflow.ExecuteActivity(ctx, AdvanceCycle, input.MoatID).Get(ctx, nil);
err != nil {
        return err
    }

    // 4. Schedule the next cycle (continues-as-new).
    return workflow.NewContinueAsNewError(ctx, CycleWorkflow, input)
}

```

4.6 The WebSocket gateway

One WebSocket connection per client per app-open, maintained for as long as the app is in the foreground. The server fans out messages per Moat. A single moat-service replica maintains a map from `moat_id` → set of WebSocket connections, but — because replicas are stateless with respect to each other — we cannot assume all members of a Moat are connected to the same replica.

Cross-replica fanout is handled via a small Redis pub-sub (or Postgres LISTEN/NOTIFY, which we use initially for simplicity). When a message arrives at any replica, the replica NOTIFYS the Moat's channel; every replica that has local WebSocket connections to that Moat's members writes the message out to them.

Yes, Postgres LISTEN/NOTIFY is an unusual message bus. We use it because it is already in the stack, scales to our message volume (Moats are 8–12 people, not Twitter), and avoids adding another piece of infrastructure. If we outgrow it, Redis pub-sub is a drop-in replacement.

4.7 Rate limits

Endpoint	Limit	Bucket / algorithm
POST /v1/moats/current/messages	20 msg/min per account	Token bucket, 20 capacity, 1 token / 3s refill.

Endpoint	Limit	Bucket / algorithm
POST /v1/moats/current/complaints	3 per cycle per account	Counter, reset at cycle boundary.
POST /v1/auth/verify	10 attempts / 10 min / device	Protects against composite-code brute-force.
GET all endpoints	120 rpm per account	Generous; meant to catch runaway clients, not tune UX.
WebSocket upgrade	1 concurrent conn / account	Reject second upgrade; disconnect newer.

5. The linkage service

A smaller program. Fewer handlers, simpler data model, different threat model. The linkage service is allowed to see plaintext — specifically, prescription email bodies — but only in transit.

5.1 Package layout

```
cmd/
└─ linkage-service/
   └─ main.go

internal/
├─ handlers/
│   ├── bootstrap.go      # POST /v1/linkage/bootstrap
│   ├── relay.go          # POST /v1/linkage/relay_prescription
│   └─ receipts.go        # GET /v1/linkage/receipts
├─ postmark/
│   ├── client.go         # Postmark API client
│   └─ health.go          # delivery webhook handler
├─ tokens/
│   └─ unity.go           # issue + verify Unity linkage tokens
├─ db/
│   ├── queries/*.sql
│   └─ generated/
├─ httputil/
│   ├── middleware.go
│   └─ errors.go
```

5.2 Handlers

Endpoint	Auth	What it does
POST /v1/linkage/bootstrap	JWT (Moat app)	Issues an opaque Unity linkage token bound to the caller's account. Returned to the app which hands it to Unity at first launch.
POST /v1/linkage/relay_prescription	Unity linkage token	Accepts a prescription envelope. Forwards to Postmark. Records only the relay_id and timestamp. Never stores body.
GET /v1/linkage/receipts	JWT (Moat app)	Returns recent receipts (relay_id, timestamp) for the user's own account. Used for the app's 'your last 7 prescriptions' UI.
POST /v1/linkage/email/set	JWT	Set / update the account's email. Calls moat-service's 'does account exist' endpoint to confirm.
POST /v1/linkage/email/remove	JWT	Delete the account's email row.
POST /v1/webhooks/postmark	Postmark signature	Delivery webhooks for audit. No body content captured — just delivery status.

5.3 The relay flow — in detail

```
POST /v1/linkage/relay_prescription
Authorization: Bearer <unity-linkage-token>
Content-Type: application/json
```

```
{
  "account_id":      "b6f0...",
  "playthrough_ref": "unity-9ab2...",
  "subject":         "A question to carry with you",
  "body_plain":      "Action: ...\\n\\nReflection: ...\\n\\nMantra: ...",
  "body_html":       null
}
```

Handler steps:

1. Validate Unity linkage token (signature + not revoked).
2. Look up the account's email via linkage DB.
If no email on file → 400 with 'no email set for this account'.
3. Rate limit: 1 relay per account per 24h.
4. Send via Postmark:


```
from:      prescriptions@moatly.[tld]
to:        <account's email>
subject:   <input subject>
body:      <input body_plain>
tag:       prescription
TrackOpens: false
TrackLinks: false
```
5. On Postmark 200:
 - a. Insert relay_receipt (relay_id, account_id, accepted_at).
 - b. In the SAME transaction, discard the body variable (Go scope).
 - c. Return 202 Accepted { relay_id, accepted_at }.
6. On Postmark error: 502 Bad Gateway. Client retries.

The body is held in a Go string variable for the duration of the handler call. After the handler returns, the variable is out of scope; Go's garbage collector eventually reclaims the backing array. We do not hold a reference, we do not log it, we do not persist it. This is the most important property of the linkage service, and §12 of this document specifies the test that verifies it.

5.4 Postmark configuration

- Separate 'transactional' server in Postmark. No marketing servers exist.
- Open-tracking and link-tracking are **off** at the server level. The tags cannot be enabled by a per-message flag that a bug could accidentally set — they are off in the server config.
- Dedicated IP. Reputation of that IP is managed by Moatly, not pooled.
- DMARC + SPF + DKIM set up on moatly.[tld]. All three strict.
- Bounce and spam-complaint webhooks fire to /v1/webhooks/postmark. We record delivery status per relay_id; no content.

6. The seven-day deletion pipeline

The Build Brief devotes an entire section (§9) to this subsystem and calls it 'the most technically sensitive' and 'treat it as the hardest subsystem to get right.' It is. This section specifies how the backend implements the three mechanisms on the server side.

6.1 The three mechanisms, restated

- **Mechanism 1 — row deletion.** Every message row is deleted after 7 days by an hourly worker; the table is VACUUM FULL'd to release pages to the OS.
- **Mechanism 2 — Megolm session rotation.** At every cycle boundary, every Moat rotates its Megolm session. Ciphertext from the old session is undecryptable because the old session state is destroyed on every member's device.
- **Mechanism 3 — backup key expiry.** Nightly Postgres snapshots are encrypted under AES-256 keys that themselves expire after 8 days. A snapshot taken on day N is decryptable until day N+8.

Each alone would be insufficient. Together they give us defence-in-depth against three different threat models: an attacker who steals the DB today (blocked by mechanism 1), an attacker who intercepts old ciphertext and later compromises a device (blocked by mechanism 2), and an attacker who obtains an old backup (blocked by mechanism 3).

6.2 Mechanism 1 — the hourly deletion worker

A Temporal workflow scheduled to run every hour. The workflow is a single activity that runs the deletion transaction.

```
func RunHourlyDeletionWorkflow(ctx workflow.Context) error {
    opts := workflow.ActivityOptions{
        StartToCloseTimeout: 10 * time.Minute,
        RetryPolicy: &temporal.RetryPolicy{
            InitialInterval: 30 * time.Second,
            MaximumAttempts: 5,
        },
    },
}
ctx = workflow.WithActivityOptions(ctx, opts)

var result DeletionResult
if err := workflow.ExecuteActivity(ctx, DeletionActivity).Get(ctx, &result); err !=
nil {
    // Failure: alert. Deletion is non-negotiable.
    workflow.GetLogger(ctx).Error("deletion-pass failed", "err", err)
    return err
}

// Emit structured audit entry.
return workflow.ExecuteActivity(ctx, WriteAuditEntry, result).Get(ctx, nil)
}

// The activity itself:
func DeletionActivity(ctx context.Context) (DeletionResult, error) {
    tx, err := db.BeginTx(ctx, &sql.TxOptions{Isolation: sql.LevelSerializable})
    if err != nil { return DeletionResult{}, err }
```

```

defer tx.Rollback()

// Snapshot the rows we're about to delete (IDs only for audit).
ids, err := db.SelectExpiredMessageIDs(ctx, tx, time.Now())
if err != nil { return DeletionResult{}, err }

n, err := db.DeleteExpiredMessages(ctx, tx, time.Now())
if err != nil { return DeletionResult{}, err }

if err := tx.Commit(); err != nil { return DeletionResult{}, err }

// Out of the transaction: VACUUM FULL on the moat_messages partition.
// (Serialisable isolation + concurrent writes make VACUUM FULL expensive;
// we partition moat_messages by month to keep the VACUUM scope bounded.)
if err := db.VacuumFullMoatMessagesCurrentPartition(ctx); err != nil {
    return DeletionResult{}, err
}

return DeletionResult{DeletedCount: n, RunID: uuid.New()}, nil
}

```

6.3 Mechanism 2 — session rotation broadcaster

At cycle boundary the cycle workflow (§4.5) broadcasts a `SessionReset` signal to all members over WebSocket. Clients create a fresh outbound Megolm session, exchange its keys with other members via 1-to-1 Olm messages (relayed by the server as opaque ciphertext), and begin encrypting under the new session. The server records the new `session_id` and discards the previous session's membership record one hour after rotation (grace window for late-arriving Olm messages).

The server holds no decryption keys. The rotation is verified by a post-rotation integrity check: the server attempts to map an old-session ciphertext to the new session and expects the operation to fail. This is done at the envelope layer, without touching message bodies.

6.4 Mechanism 3 — backup key expiry

RDS snapshots are encrypted by AWS KMS. We configure KMS to schedule deletion of the snapshot encryption key 8 days after creation. After 8 days, the snapshot bytes remain but cannot be decrypted.

Implementation note — KMS has a minimum scheduled-deletion window of 7 days. We configure 8 days to give ourselves a 1-day margin. ADR-008 (brief §C) addresses this constraint.

```

# Daily at 02:00 UTC:
# 1. Take an RDS snapshot: moatly-nightly-YYYY-MM-DD.
# 2. Create a KMS customer-managed key: k-snap-YYYY-MM-DD.
# 3. Re-encrypt the snapshot under k-snap-YYYY-MM-DD.
# 4. Schedule k-snap-YYYY-MM-DD for deletion in 8 days.
# 5. Log (k-snap-YYYY-MM-DD, scheduled_deletion_at) to the audit log.
# Reconciliation job (weekly):
# - For every k-snap-* created >8 days ago:
#   verify the KMS key status is PendingDeletion or Deleted.
#   If not, alert – manual deletion may have been cancelled.

```

6.5 The deletion audit log

Every run of the hourly worker writes a row to `deletion_audit`. This table is append-only and retained for 2 years (brief §7.2).

```
CREATE TABLE deletion_audit (
  run_id          UUID PRIMARY KEY,
  ran_at          TIMESTAMPTZ NOT NULL,
  deleted_count   INTEGER NOT NULL,
  vacuum_duration_ms BIGINT NOT NULL,
  mechanism       TEXT NOT NULL CHECK (mechanism IN
    ('hourly', 'cycle_rotation', 'snapshot_key_expiry')),
  success         BOOLEAN NOT NULL,
  notes          TEXT
);

-- Not deletable. Permission on this table grants INSERT only to the
-- deletion role. No DELETE, no TRUNCATE, no ALTER.
```

6.6 Failure handling

Failure	Detection	Response
Hourly worker fails	Temporal retry exhausted.	Page on-call. Sev-1. If not recovered in 6h, manual DELETE via runbook.
VACUUM FULL hangs	Activity timeout.	Abort; retry on next hour. Persistent failure → incident report.
SessionReset broadcast fails	Per-client ack metric below threshold.	Extend grace window; retry broadcast. Clients catch up on reconnect.
KMS deletion cancelled	Reconciliation alerts.	Re-schedule immediately. Incident report published.
Deletion audit write fails	Transaction abort.	Treat as total failure of the run — do not leave deletion un-audited.

Every deletion failure is Sev-1, regardless of business impact.

The Build Brief (§9.6) is explicit: 'If the deletion pipeline fails silently, Moatly has lied to its users.' A deletion failure does not delay shipping. It does not get triaged during business hours. It pages the on-call. If the on-call cannot resolve it, the founder is woken up.

7. Data model and retention

The schemas below are the complete DDL for both services. They extend the brief's §7.1 with the full set of tables we actually need. Every table has an explicit retention rule. Every column that could contain PII is called out in §7.5.

7.1 moat-service schema

```
-- Accounts: opaque. No PII.
CREATE TABLE accounts (
  account_id      UUID PRIMARY KEY,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  covenant_hash   BYTEA NOT NULL,
  status          TEXT NOT NULL DEFAULT 'active'
                  CHECK (status IN ('active','suspended','deleted'))
);

-- Devices: 1:1 with accounts for hardware; N:1 for virtual.
CREATE TABLE devices (
  device_id       UUID PRIMARY KEY,
  account_id      UUID NOT NULL REFERENCES accounts(account_id),
  hw_attestation_pubkey BYTEA NOT NULL,
  factory_cert    BYTEA NOT NULL,
  is_virtual      BOOLEAN NOT NULL DEFAULT false,
  paired_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  tombstoned_at   TIMESTAMPTZ,
  revoked_at      TIMESTAMPTZ,
  last_handshake_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX idx_devices_pubkey ON devices (hw_attestation_pubkey);
CREATE INDEX idx_devices_account ON devices (account_id);

-- Moats.
CREATE TABLE moats (
  moat_id         UUID PRIMARY KEY,
  created_at      TIMESTAMPTZ NOT NULL,
  status          TEXT NOT NULL DEFAULT 'forming'
                  CHECK (status IN ('forming','active','dissolved')),
  cycle_number    INTEGER NOT NULL DEFAULT 0,
  cycle_ends_at   TIMESTAMPTZ NOT NULL,
  current_session_id TEXT NOT NULL
);

-- Membership is session-scoped. Only current + previous retained.
CREATE TABLE moat_membership (
  moat_id         UUID NOT NULL REFERENCES moats(moat_id),
  account_id      UUID NOT NULL REFERENCES accounts(account_id),
  pseudonym       TEXT NOT NULL,
  joined_session  INTEGER NOT NULL,
  left_session     INTEGER,
  PRIMARY KEY (moat_id, account_id)
);
```

```
-- Messages. 7 days, partitioned monthly for VACUUM scope control.
CREATE TABLE moat_messages (
  message_id      UUID PRIMARY KEY,
  moat_id         UUID NOT NULL,
  cycle_number    INTEGER NOT NULL,
  session_id      TEXT NOT NULL,
  message_index   INTEGER NOT NULL,
  type            TEXT NOT NULL CHECK (type IN
    ('whisper', 'major_statement', 'witness', 'presence')),
  ciphertext      BYTEA NOT NULL,
  parent_message_id UUID,
  posted_at       TIMESTAMPTZ NOT NULL,
  expires_at      TIMESTAMPTZ NOT NULL,
  deleted_at      TIMESTAMPTZ
) PARTITION BY RANGE (posted_at);

CREATE INDEX idx_mm_expiry ON moat_messages (expires_at) WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX idx_mm_dedupe ON moat_messages (session_id, message_index);

-- Complaints: metadata only. Never body.
CREATE TABLE complaints (
  complaint_id    UUID PRIMARY KEY,
  moat_id         UUID NOT NULL REFERENCES moats(moat_id),
  complainant_id  UUID NOT NULL REFERENCES accounts(account_id),
  target_id       UUID NOT NULL REFERENCES accounts(account_id),
  reason          TEXT NOT NULL CHECK (reason IN
    ('advice', 'targeting', 'commercial', 'off_covenant', 'abuse')),
  cycle_number    INTEGER NOT NULL,
  filed_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  resolved_at     TIMESTAMPTZ,
  resolution      TEXT CHECK (resolution IN
    ('no_action', 'reassigned', 'warned', 'banned'))
);
CREATE INDEX idx_cmp_target ON complaints (moat_id, target_id, reason, cycle_number);
CREATE INDEX idx_cmp_complainant ON complaints (complainant_id, filed_at);

-- Placement queue (FIFO): account IDs waiting to be placed into a Moat.
CREATE TABLE placement_queue (
  account_id      UUID PRIMARY KEY REFERENCES accounts(account_id),
  queued_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  country_hint    TEXT
);

-- Megolm sessions: server stores session identifier + metadata, no keys.
CREATE TABLE megolm_sessions (
  session_id      TEXT PRIMARY KEY,
  moat_id         UUID NOT NULL REFERENCES moats(moat_id),
  cycle_number    INTEGER NOT NULL,
  created_at      TIMESTAMPTZ NOT NULL,
  retired_at      TIMESTAMPTZ,           -- set at cycle boundary rotation
  purged_at       TIMESTAMPTZ           -- set 1h after retirement
);

-- Deletion audit – append-only, 2-year retention.
```

```
CREATE TABLE deletion_audit (
  run_id          UUID PRIMARY KEY,
  ran_at          TIMESTAMPTZ NOT NULL,
  deleted_count   INTEGER NOT NULL,
  vacuum_duration_ms BIGINT NOT NULL,
  mechanism       TEXT NOT NULL,
  success         BOOLEAN NOT NULL,
  notes          TEXT
);
```

7.2 linkage-service schema

```
-- Email addresses, independent of Moat account row. GDPR-friendly.
CREATE TABLE account_email (
  account_id      UUID PRIMARY KEY,
  email_address   TEXT NOT NULL,
  verified_at     TIMESTAMPTZ NOT NULL,
  set_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  unsubscribed_at TIMESTAMPTZ
);
CREATE INDEX idx_email_address ON account_email (lower(email_address));

-- Unity linkage tokens.
CREATE TABLE unity_tokens (
  token_id        UUID PRIMARY KEY,
  account_id      UUID NOT NULL,
  issued_at       TIMESTAMPTZ NOT NULL,
  expires_at      TIMESTAMPTZ NOT NULL,
  revoked_at      TIMESTAMPTZ
);

-- Relay receipts. Body NEVER stored; only the receipt.
CREATE TABLE relay_receipts (
  relay_id        UUID PRIMARY KEY,
  account_id      UUID NOT NULL,
  accepted_at     TIMESTAMPTZ NOT NULL,
  postmark_id     TEXT, -- Postmark's own message identifier
  delivery_status TEXT CHECK (delivery_status IN
    ('pending', 'delivered', 'bounced', 'complaint', 'failed'))
);
-- 30-day retention: receipts retained to show the user "last 7 delivered",
-- then pruned.
CREATE INDEX idx_receipts_account_time ON relay_receipts (account_id, accepted_at
DESC);
```

7.3 Retention policy (complete)

Data	Retention	Mechanism
moat_messages.ciphertext	7 days	Hourly worker + VACUUM FULL (mechanism 1).
moat_messages in DB backups	≤ 8 days	Snapshot key expiry (mechanism 3).

Data	Retention	Mechanism
megolm_sessions (retired)	1 hour after retirement	Purged after grace window.
moat_membership (left)	2 cycles	Kept briefly so late-arriving messages can still resolve sender pseudonyms.
complaints	90 days after resolution	For detecting serial complaint abuse.
relay_receipts	30 days	Pruned by nightly job.
account_email	Until user removes	Deletable independently via DELETE /v1/accounts/me/email.
devices (revoked)	Tombstoned 14d, then 0d (revoked)	Two-stage to support the 'found my old device' case.
deletion_audit	2 years	Proof-of-deletion, required for transparency reports.
placement_queue	Until placed	Row deleted at placement.
unity_tokens	Expire 1y, pruned 30d later	Long-lived but rotatable.

7.4 What is never stored anywhere

- Message plaintext. Ever. Not in logs, not in metrics, not in traces, not in breakglass, not in a debug endpoint.
- Prescription email bodies. They exist in linkage-service process memory briefly, then are gone.
- IP addresses on a per-request basis. Stripped at the WAF before downstream logging.
- User-agent strings in their full form. Binned to (os, major-version).
- Real names of any user. There are no name columns.

7.5 PII audit — columns that could contain personal data

Column	Service	Why it exists
account_email.email_address	linkage	Required for prescription delivery. Separately deletable.
relay_receipts.postmark_id	linkage	Lets us correlate with Postmark delivery webhooks. Not PII in itself.
devices.hw_attestation_pubkey	moat	Not personal — describes a device. But strongly correlated with an account.
complaints.complainant_id, target_id	moat	Opaque account IDs — not names, but can be correlated if the server is compromised.

The PII audit is run quarterly. Any new column that could hold personal data must either be added to this table or the migration is rejected.

8. Full API surface

Every public endpoint. Grouped by service; auth requirement annotated.

8.1 moat-service — auth

Method	Path	Auth	Purpose
POST	/v1/auth/challenge	None	Issue a server nonce for device handshake.
POST	/v1/auth/verify	None	Exchange composite identity code + signature for a JWT.
POST	/v1/auth/refresh	JWT	Refresh a JWT if still valid.
POST	/v1/auth/softauth/start	None	(REMOVED at Phase 3 T-0.) Soft-auth email step.
POST	/v1/auth/softauth/verify	None	(REMOVED at Phase 3 T-0.) Soft-auth completion.

8.2 moat-service — accounts and devices

Method	Path	Auth	Purpose
POST	/v1/accounts	None (first-pair only)	Create account bound to a device attestation.
DELETE	/v1/accounts/me	JWT	Delete account. Triggers purge workflow.
POST	/v1/devices/enrol	JWT	Enrol an additional device (replacement).
POST	/v1/devices/tombstone	JWT	Tombstone a device for 14d before revocation.
POST	/v1/devices/revoke	JWT	Immediate revocation (stolen device).

8.3 moat-service — moats, messages, complaints

Method	Path	Auth	Purpose
POST	/v1/moats/join	JWT	Add account to placement queue.
GET	/v1/moats/current	JWT	Return current Moat state (members, session_id, cycle).
POST	/v1/moats/current/messages	JWT	Post an encrypted message.

Method	Path	Auth	Purpose
T			
GET	/v1/moats/current/messages	JWT	Backfill — messages since a given index.
POST	/v1/moats/current/ws/ticket	JWT	Issue a short-lived WebSocket upgrade ticket.
WS	/v1/moats/current/stream?ticket=...	Ticket	WebSocket stream of new messages.
POST	/v1/moats/current/session/reset	Internal	System endpoint; triggered by cycle workflow.
POST	/v1/moats/current/complaints	JWT	File a complaint.
GET	/v1/moats/current/complaints/mine	JWT	List own complaints.

8.4 moat-service — firmware and ops

Method	Path	Auth	Purpose
GET	/v1/firmware/manifest/latest	None	Signed manifest for latest firmware.
GET	/v1/firmware/manifest/:version	None	Signed manifest for a specific version (rollback protection).
GET	/v1/ops/deletion/status	Ops auth	Deletion pipeline status for the status page.
GET	/v1/ops/metrics	Internal	OpenTelemetry scrape endpoint.
GET	/v1/ops/health	None	ALB health check.

8.5 linkage-service endpoints

Method	Path	Auth	Purpose
POST	/v1/linkage/bootstrap	JWT (Moat)	Issue Unity linkage token.
POST	/v1/linkage/relay_prescription	Unity linkage token	Relay prescription via Postmark.
GET	/v1/linkage/receipts	JWT	List recent relay receipts.
POST	/v1/linkage/email/set	JWT	Set account email.
POST	/v1/linkage/email/remove	JWT	Remove account email.

Meth od	Path	Auth	Purpose
POST	/v1/webhooks/postmark	Postmark sig	Delivery webhooks.

8.6 Request and response shapes — example

For brevity, one representative example. Full OpenAPI schemas ship in the shared schemas package.

```
POST /v1/moats/current/messages
Host: api.moatly.[tld]
Authorization: Bearer <jwt>
Content-Type: application/x-megolm-group-message
X-Envelope-Version: 1
```

<binary Megolm envelope; outer format is protobuf:

```
message MoatMessageEnvelope {
  string session_id      = 1;
  int32  message_index   = 2;
  string message_type    = 3;      // 'whisper' | 'major_statement' | ...
  int64  posted_at_ms    = 4;      // client-side timestamp
  string parent_message_id = 5;    // optional, for witnesses
  bytes  ciphertext       = 6;      // Megolm ciphertext body
  bytes  auth_tag         = 7;
}>
```

— Response on success —

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{
  "message_id": "0f9c...",
  "posted_at": "2026-04-20T14:02:11.437Z",
  "expires_at": "2026-04-27T14:02:11.437Z"
}
```

— Possible error responses —

```
400 { "code": "malformed_envelope" }
401 { "code": "invalid_jwt" }
403 { "code": "wrong_cycle" }      - session_id is from a previous cycle
409 { "code": "duplicate_index" }  - (session_id, message_index) seen
413 { "code": "too_large" }
429 { "code": "rate_limited", "retry_after_s": 3 }
503 { "code": "cycle_rotating" }  - cycle boundary; retry in a few sec
```

9. Server-side cryptography

The server is blind to Moat content. That property is load-bearing. This section specifies what the server does with cryptographic material (attestation, JWTs, session identifiers, backup keys) so that the blindness is provable.

9.1 Device attestation verification

At `/v1/accounts` (new account) or `/v1/devices/enrol` (replacement), the client sends an attestation payload (§4.4 of Phase 3 breakdown). The server verifies:

- **Factory certificate chains to Moatly's pinned factory CA** — the CA's public key is compiled into `moat-service` as a constant.
- **The device_serial in the cert is not in the revocation list** — revocation list is a Postgres table loaded at startup and refreshed every 5 minutes.
- **The signature on the challenge matches under hw_attestation_pubkey** — using Ed25519. Standard signature verification.

On success, the account + device rows are written in a single transaction. Enrolment is idempotent: re-enrolling the same device against the same account is a no-op.

9.2 Composite identity code — server-side treatment

The composite identity code (Phase 3 §6) is a value the server records and uses for statistical anomaly detection, not for cryptographic verification. Cryptographic verification is the device-signature step (§9.1 of this document).

For each successful `/v1/auth/verify`, the server records a `code_digest` (SHA-256 of the submitted composite code). Codes beyond the last 90 days are pruned. We never reconstruct behavioural fingerprints from codes — we cannot, because the HMAC is keyed by a secret the server does not have. We only ask 'did this account previously submit codes with a similar distribution?' as a soft anomaly signal.

Concretely: an account that suddenly submits codes with much higher Hamming distance from its historical average is flagged for ops review. No action is taken automatically — this is a human-triage signal only.

9.3 JWT issuance

Every successful handshake yields a JWT. Short-lived (15 min). Claims:

```
{
  "sub":      "<account_id>",      // subject
  "aud":      "moat.api.moatly",   // audience
  "iss":      "moat.auth.moatly",  // issuer
  "iat":      <unix-seconds>,
  "exp":      <iat + 900>,         // 15 minutes
  "moat_id":  "<moat_id>",
  "cycle_number": <int>,           // prevents cross-cycle writes
  "device_id": "<device_id>",
  "jti":      "<uuid>"             // for revocation
}
```

Signing algorithm: EdDSA (Ed25519). Key lives in AWS KMS.

Public key is published at GET /v1/auth/jwks (for internal services).

9.4 Server-side vodozamac usage

The server does *not* decrypt. It uses vodozamac only to parse Megolm envelope metadata:

- Extract session_id from the envelope — so we can verify it matches the Moat's current session.
- Extract message_index — so we can dedupe and detect replays.
- Verify the outer MAC — so a malformed envelope is rejected before we write to the database.

We wrap vodozamac's Go bindings (generated via UniFFI, same approach as the mobile side) and expose only those three operations. The wrapper refuses to expose anything involving group or inbound session keys. A hostile change to the wrapper that attempted to expose decryption would be visible in code review; the wrapper is a tiny file.

9.5 JWT key rotation

The JWT signing key is rotated quarterly. The rotation overlaps: for a 24-hour window we accept JWTs signed by either the old or new key. After the window, only the new key is accepted. Rotation cadence matches the brief's KMS posture (§6.3, 'server TLS key, rotates 90 days').

9.6 Backup encryption

Detail in §6.4. Here we note that the backup key rotation is *not* just a scheduling concern — every daily key is a separate KMS customer-managed key, tagged with its date of creation, with scheduled deletion set at creation time. This makes every day's backup independently auditable: the weekly reconciliation job (§6.4 fragment) asserts that every key older than 8 days is PendingDeletion or Deleted.

10. Observability

10.1 What we emit

The entire observability posture is shape-only: metrics, traces, and structured logs that contain no user content. Three pillars.

10.1.1 Metrics

- Request rate per endpoint, per method, per response status.
- Latency percentiles (p50, p95, p99) per endpoint.
- WebSocket concurrent connections.
- Deletion job success / failure counters, duration histograms.
- Cycle rotation success / failure counters.
- Rate-limit rejection counters per rule.
- Postmark delivery status counters.
- Postgres connection pool utilisation, slow query counters.
- Temporal workflow success / failure counters.

10.1.2 Traces

- One OpenTelemetry trace per HTTP request.
- Span attributes include endpoint, HTTP status, account_id (opaque). They **do not** include request bodies, response bodies, SQL parameters containing ciphertext, or any header beyond an allow-list (method, path, user-agent bucket).
- Span names are fixed constants, not derived from request data (derived names can leak PII by accident).

10.1.3 Logs

- Structured JSON via a logger that **refuses** to accept fields named body, content, plaintext, or text.
- Log levels INFO and above to Grafana Loki. DEBUG is process-local only and disabled in production.
- Retention: 30 days.
- IP and full user-agent stripped at the edge.

10.2 What we deliberately do not emit

- Request body content.
- Response body content.
- SQL parameters when they contain BYTEA values — we log the column name and byte length instead.
- User-behaviour-derived metrics (e.g. 'messages per user per day'). Operational metrics only.
- Any field that could be reconstructed into content.

10.3 Enforcement

- Static analysis: an AST rule in CI flags any `slog.String('body', ...)` or equivalent. PR fails.
- Runtime: the logger has an allowlist of field names; unknown field names produce a 'field elided' log entry (shape-only) and an ops alert.
- Proxy testing: CI runs every integration test behind a filtering proxy whose allowlist is compared to actual outbound hosts.

11. Security and operational posture

11.1 Production access

- **SSH to any production host requires a FIDO2 hardware security key.** Password-based SSH is disabled.
- AWS console access requires MFA via a FIDO2 key.
- Four-eyes rule on production deploys: one engineer opens the PR, a different engineer approves it, CI deploys after approval.
- Break-glass access is logged to an append-only S3 bucket. Break-glass use is reviewed within 24 hours by the security engineer.

11.2 Secrets management

- Runtime secrets: AWS Secrets Manager, encrypted under customer-managed KMS keys.
- In-repo config: SOPS with KMS. No plain-text secret ever committed.
- Secrets rotation: quarterly for long-lived secrets (Postmark API key, signing keys); per-deploy for short-lived (JWT signing key).
- CI secrets: scoped to specific pipelines; repo-wide secrets are forbidden.

11.3 Lawful orders

The Build Brief (§6.5) states: 'Quarterly public transparency reports — number of lawful orders received, number resisted, number complied with (we expect the last figure to always be 0 for Moat content, because we cannot comply).' The backend's role in this is to make the claim true.

- When a lawful order arrives: legal logs it in the tracker (separate system, not in the service DB).
- If the order requests content: we do not hold it. The response is a technical explanation + copy of our public architecture.
- If the order requests metadata (account existence, device pairing date): we respond with what we have. This is minimal.
- Every order and our response is summarised in the next quarterly transparency report.

The architecture is the defence.

We do not rely on legal refusal to protect Moat content. The architecture's blindness is the defence. When a lawful order arrives asking for messages, our response is 'these do not exist in a form we can provide — here is the open-source architecture that explains why.' This is the posture the brief calls for and the codebase supports.

11.4 Incident response

- Sev-1: any deletion pipeline failure, any credible data exposure, any compromise of production credentials. Paged immediately. Founder notified.
- Sev-2: elevated error rates, degraded SLO, non-urgent security findings. Next business day.
- Sev-3: operational weirdness with no user impact. Sprint-prioritised.

- **Post-mortems** are published within 30 days on the portal's /incidents page per Build Brief §12.4.

11.5 Deploy process

1. PR opened against main. CI runs: lint, unit, integration, privacy gate.
2. Review by a different engineer. Approvals tracked in the repo.
3. Merge to main.
4. Release pipeline:
 - a. Build image, push to ECR.
 - b. Deploy to staging cluster; run E2E suite.
 - c. Manual gate: SRE or lead approves production deploy.
 - d. Deploy to prod, canary (10%) for 15 min, then full rollout.
5. Post-deploy: smoke tests, dashboard review for error-rate bumps.
6. Any production-data migration requires a separate approval.

11.6 Backups and disaster recovery

- Nightly RDS snapshots, encrypted under daily-rotated KMS keys (8-day life).
- Read replica in eu-west-1 (Dublin) for failover.
- RTO target: 30 minutes for a region failover.
- RPO target: 5 minutes (synchronous replication for Multi-AZ; async for cross-region).
- Quarterly DR exercise: simulate a region failure in staging, verify failover completes in RTO.

12. Testing

12.1 The pyramid

Level	Tool	Scope	Target count (end of Phase 2)
Unit	go test	Handler logic, domain packages, crypto wrappers.	~1,500 tests
Property	gopter or rapid	Megolm envelope parsing, JWT claims, dedup logic.	~25 properties, 10k iters each
Integration	go test + containerised Postgres + Temporal	DAO + workflows against real infra.	~200 tests
E2E	Custom Go harness + simulated clients	Full flow: handshake, post, fanout, cycle rotate.	~25 flows
Adversarial	Bespoke	No-plaintext-anywhere; deletion adversarial; blind-routing.	See §12.2
Static	staticcheck + custom AST rules	No body logging, no layer violations.	Continuous

12.2 Adversarial tests

Seven tests that try to break the server's core promises. Each is run on every main-branch deploy.

Test	What it proves
BLIND-1 — Log scan for content	Post 100 messages with known magic strings (as ciphertext). Scan every log sink and trace export for any occurrence of a post-decryption fragment. Expect zero matches.
BLIND-2 — DB row scan	After 100 messages posted, dump every row of moat_messages. Verify no column contains the known magic strings (i.e. the server truly has only ciphertext, no decrypted caches).
BLIND-3 — Memory dump	Core-dump a moat-service process after handling a message. Scan the dump for any decrypted fragment. Expect zero.
DELETE-1 — After 8 days	Post 1,000 messages with known timestamps. Fast-forward 8 days. Run the deletion pipeline. Dump the DB file. Expect zero matches for the 1,000 messages.
DELETE-2 — Backup key expiry	Take a backup. Advance KMS key scheduled-deletion to expiry. Attempt to decrypt the backup. Expect failure.
ROUTE-1 — Cross-Moat	Authenticate as user A, attempt to POST a message with a session_id from Moat B. Expect 403.
ROUTE-2 — Replay	Post a message. Re-post the same envelope. Expect 409 duplicate_index;

Test	What it proves
	single row in DB.

12.3 Load testing

- k6 runs against staging pre-release. Target: 10,000 concurrent WebSocket connections; 500 messages/sec steady-state; no degradation for 60 minutes.
- Deletion-pipeline stress test: 10,000,000 messages aged out in a single run; must complete in under 30 minutes.
- Cycle-rotation at scale: 1,000 simultaneous Moats rotating. Broadcast fanout latency p95 < 2s.

13. Implementation plan

The backend plan spans 10 months, mapped to the Build Brief's Phase 0–4 structure. Team: 2 Go engineers (full-time), 1 SRE (full-time from month 3), 1 security engineer (fractional ramping to full-time at audit).

13.1 Phase mapping

Phase	Months	Backend milestone
0	1–1.5	Repo scaffold, CI, infra-as-code, threat model, crypto design review, ADRs 1–13 approved.
1	1.5–4.5	linkage-service live. /v1/accounts, /v1/linkage/* endpoints. Email relay working. JWT issuance (soft-auth path). No Moats yet.
2	4.5–7	moat-service live. Full Moats functionality. Megolm envelope parsing. Deletion pipeline. Cycle workflow. WebSocket gateway. Complaints. Soft-auth used by 50-person alpha.
3	7–10	Device attestation endpoints. Firmware manifests. Composite code. Soft-auth decommissioned at T-0.
4	10–10.5	Public-beta hardening. External audit remediations. Public repo activation. First transparency report.

13.2 Weekly plan — Phase 0 (weeks 1–6)

W	Backend engineer A (foundations)	Backend engineer B (service skeletons)
1	Repo scaffold. CI (lint, unit, SAST, SBOM).	Threat model doc. ADR-1 through ADR-5 drafts.
2	Terraform for AWS EU: VPC, ALB, RDS, ECS.	moat-service skeleton. Health endpoint.
3	Temporal self-hosted on ECS. DR pattern.	linkage-service skeleton. Postmark integration stub.
4	sqlc + migrations. First Postgres schema.	JWT signing + verification. KMS-backed signer.
5	Observability: OTel, Loki, Grafana Cloud.	Privacy logger. AST rules for no-body.
6	End of Phase 0: crypto design reviewed by ext.	ADRs approved. First commit lands in public repo.

13.3 Weekly plan — Phase 1 (weeks 7–18)

W	Backend engineer A	Backend engineer B
7	/v1/accounts handler + tests.	Account email tables + /v1/linkage/email/*.
8	/v1/auth/softauth handlers (time-bounded).	Unity tokens + /v1/linkage/bootstrap.
9	JWT claims + refresh endpoint.	/v1/linkage/relay_prescription + Postmark.

W	Backend engineer A	Backend engineer B
10	Rate limiting middleware.	Relay receipts + /v1/linkage/receipts.
11	/v1/accounts/me DELETE + purge workflow.	Postmark delivery webhook.
12	Observability dashboards.	Contract tests against mobile app mocks.
13	Backfill: integration test against mobile alpha.	Backfill: integration test against Unity fixtures.
14	Bug-burn with 10-person dogfood.	Bug-burn with 10-person dogfood.
15	Phase 1 gate prep.	Phase 1 gate prep.
16	(Reserved for Phase 1 gate remediation.)	Buffer.
17	Start of Phase 2 scaffold work.	Start of Phase 2 scaffold work.
– 18		

13.4 Weekly plan — Phase 2 (weeks 17–28)

W	Backend engineer A	Backend engineer B
17	Moat schema + placement queue.	vodozamac Go wrapper for envelope parsing.
18	/v1/boats/join + placement worker.	/v1/boats/current + GET messages.
19	POST message handler + dedupe logic.	WebSocket gateway + ticket exchange.
20	Broadcast fanout via LISTEN/NOTIFY.	Cycle workflow in Temporal.
21	Deletion workflow + audit log.	Session-reset broadcaster.
22	Complaints handlers + thresholds.	Serial-complaint detection + review queue.
23	Adversarial tests BLIND-*, DELETE-*, ROUTE-*.	Load tests to 10k concurrent WS.
24	Backup key expiry automation + reconciliation.	Cycle rotation integration test end-to-end.
25	Phase 2 gate prep — alpha deploy.	Phase 2 gate prep — alpha deploy.
26	50-person alpha bug-burn.	50-person alpha bug-burn.
27	First transparency report (Q? to-date).	First transparency report — metrics compilation.
28	Phase 2 exit.	Phase 2 exit.

13.5 Weekly plan — Phase 3 (weeks 29–40)

W	Backend engineer A	Backend engineer B
29	Device attestation verification.	Factory CA setup + revocation list.
30	/v1/accounts with device attestation.	/v1/devices/enrol + tombstone + revoke.
31	/v1/auth/challenge + /v1/auth/verify.	Composite code recording + anomaly

W	Backend engineer A	Backend engineer B
		detection.
32	Firmware manifest endpoint + signing.	JWKS endpoint + quarterly rotation.
33	Soft-auth: T-30 announcements start.	Soft-auth: T-30 banner feature on backend.
34	Soft-auth: T-14 deprecation header.	Soft-auth: 410 Gone responses.
35	Soft-auth: T-0 removal PR drafted.	Phase 3 alpha deploy for 100-person cohort.
36	100-person bug-burn.	100-person bug-burn.
37	Soft-auth T-0 cutover: PR merged, release shipped.	Backend soft-auth endpoints removed.
38	Soft-auth T+7 cleanup: endpoint code deleted.	Soft-auth T+7 cleanup: endpoint code deleted.
39	External audit window begins.	External audit window begins.
40	Phase 3 exit.	Phase 3 exit.

13.6 Phase 4 (weeks 41–44)

- External audit remediation of any findings.
- Public repo activation as primary remote (was mirrored).
- First fully-public transparency report.
- TestFlight / Play open beta support.
- Operational runbook completion.

14. Open questions

Question	Owner	Needed by	Default if no answer
Confirm AWS region strategy: Frankfurt primary, Dublin replica.	Lead + SRE	Phase 0	Per brief §4.2 — proceed with it.
Temporal self-hosted or Cloud?	Lead + SRE	Phase 0	Self-hosted per §2.4 recommendation.
Rate limit exact values — ratified by product?	Product + lead	Phase 1	Use §4.7 as-is; tighten if abused.
Complaint review queue — human staffing model.	Founder + ops	Phase 2	1 reviewer, 8h/week allocation.
Postmark dedicated IP — warm-up plan.	SRE	Phase 1	Follow Postmark's standard warm-up; 14-day ramp.
Factory CA private-key custody.	Security + founder	Phase 3	Two-of-two offline custody (security engineer + founder).
Confirm 90-day complaint retention.	Legal + founder	Phase 2	Per brief §7.2 — 90 days from resolution.
Deletion audit retention — 2 years or longer?	Legal	Phase 2	2 years per brief §7.2.
Cross-region failover runbook — who runs it?	SRE + lead	Phase 4	On-call SRE; founder approval for failover trigger.

15. Post-launch

15.1 What the ongoing shape looks like

After public beta the backend enters a maintenance mode. The team contracts (per brief §11.1) from ~9.5 FTE to ~6. The recurring work is:

- Quarterly transparency report preparation — metric compilation by SRE, narrative by lead + founder.
- Security updates — dependency upgrades, vodozemacs version bumps with audit.
- Incident response when incidents occur. Post-mortems published within 30 days.
- Cohort management — watching placement queue velocity, adjusting capacity.
- Operational tuning — database vacuuming, Postgres major version upgrades, KMS key lifecycle.

15.2 What will be tempting and should be resisted

The biggest pressure is 'just one feature.'

A profile. A search. A 'discover other Moats.' A friend-link. A trending topic. A notification of something other than a Major Statement. Each of these is a request that, in isolation, 'wouldn't really' violate any principle. Each one, in aggregate, turns Moatly into something else. The backend's posture has to be 'no, because of §2 of the brief' — repeatedly, cheerfully, and without hedging.

15.3 Long-running concerns

- Postgres partition maintenance — monthly partitioning of `moat_messages` means dropping old partitions is automatic; new partitions are created 30 days ahead by a scheduled job.
- vodozemacs upgrades — every 6 months a review of the vodozemacs changelog for relevant changes. Security engineer drives.
- Temporal version upgrades — annual.
- KMS key rotation cadence — per brief §6.3: JWT keys 90 days, backup keys daily with 8-day life, attestation CA keys never (device lifetime).
- Quarterly PII audit — the security engineer reviews every new column added in the quarter. Any PII-possible column gets added to §7.5 of this document.

15.4 Closing

The backend is small on the surface. Two services, a shared database, a workflow engine, a handful of external integrations. What makes it non-trivial is the set of promises it has to keep at the same time: server-blind, provably destroyed, consistently available, visibly open. The plan in §13 is long because getting those four right, together, takes time.

Keep it boring. Keep it small. Keep it open. The moment the backend becomes interesting — in the sense of novel, clever, proprietary — is the moment it has started to work against the product.

— *End of Backend Technical Breakdown* —

Moatly · Backend technical breakdown · April 2026 · v0.1